DATA 612 · Statistical Programming in R · Week 1

1.4 Introduction to R Concepts

Data types, data structures, indexing, functions, and special values.

1. Values and Objects

R works with **values**. For example:

```
10
3.14
"Hello"
TRUE
```

We can store a value in an **object** using the assignment operator `<-`:

```
x <- 10
name <- "John"
passed <- TRUE
```

Anything after `#` on a line is a **comment**. R ignores it:

```
x <- 10   # store the number 10 in x
```

In this course we use `<-` for assignment. We use `=` for named function arguments, which we will meet shortly.

We can then use these objects later:

```
x
name
passed
```

---

2. Basic Data Types

Some of the most common data types in R are:

Type	Examples	Description
Numeric	`5`, `3.14`, `-10`	Numbers
Integer	`5L`, `10L`	Whole numbers explicitly stored as integers
Character	`"Hello"`, `"John"`	Text
Logical	`TRUE`, `FALSE`	Boolean values
Complex	`2 + 3i`	Complex numbers

Examples:

```
x <- 10
y <- "Hello"
z <- TRUE
```

We can investigate objects using several functions:

```
typeof(x)
class(x)
mode(x)
length(x)
str(x)
```

For example:

```
typeof(10)
# "double"

typeof("Hello")
# "character"

typeof(TRUE)
# "logical"
```

Numeric vs. Integer

By default, R stores ordinary numbers as doubles:

```
typeof(5)
# "double"
```

To explicitly create an integer, add `L`:

```
typeof(5L)
# "integer"
```

Both are considered numeric:

```
is.numeric(5)
# TRUE

is.numeric(5L)
# TRUE
```

Useful type-checking functions include:

```
is.numeric()
is.integer()
is.double()
is.character()
is.logical()
is.complex()
```

R also provides conversion functions:

```
as.numeric()
as.integer()
as.character()
as.logical()
as.complex()
```

For example:

```
as.character(25)
# "25"

as.numeric("25")
# 25

as.logical(1)
# TRUE
```

---

3. Vectors

A **vector** is one of the most fundamental data structures in R.

A vector contains a sequence of values, normally of the **same basic type**.

We use `c()` ("combine") to create a vector:

```
x <- c(10, 20, 30, 40)

x
# [1] 10 20 30 40
```

The vector contains four elements:

```
length(x)
# 4
```

We can create vectors of different types:

```
numbers <- c(10, 20, 30)

names <- c("John", "Sara", "David")

answers <- c(TRUE, FALSE, TRUE)
```

Useful functions for exploring vectors include:

```
length(x)
typeof(x)
class(x)
str(x)

head(x)
tail(x)

unique(x)
duplicated(x)

sort(x)
rev(x)
```

Vectorized arithmetic

R applies arithmetic to every element of a vector:

```
x <- c(10, 20, 30, 40)

x * 2
# 20 40 60 80

x + 1
# 11 21 31 41

x / 10
# 1 2 3 4
```

If two vectors have different lengths, R **recycles** the shorter one. We will look at this more carefully in the lab.

Useful numerical summaries include:

```
sum(x)
mean(x)
median(x)
min(x)
max(x)
range(x)
var(x)
sd(x)
```

---

4. What Happens When Types Are Mixed?

An atomic vector normally contains one type.

Consider:

```
x <- c(1, 2, "hello")
```

R cannot keep some elements numeric and one character in the same atomic vector, so it converts them to a common type:

```
x
# "1" "2" "hello"

typeof(x)
# "character"
```

This automatic conversion is called **coercion**.

Another example:

```
c(TRUE, 5)
```

R converts `TRUE` to a numeric value:

```
# 1 5
```

A useful simplified coercion hierarchy is:

```
logical → integer → double → character
```

---

5. Lists

What if we actually want one object to contain values of **different types**?

We can use a **list**.

A list is a data structure that can contain elements of different types and even different structures.

```
x <- list(
  25,
  "John",
  TRUE
)

x
```

Unlike an atomic vector, the elements do not need to have the same type.

We can also name the elements:

```
student <- list(
  name = "John",
  age = 25,
  grades = c(90, 85, 95),
  passed = TRUE
)
```

Useful functions include:

```
length(student)
names(student)
str(student)
is.list(student)
```

Lists are extremely flexible: an element of a list can itself be a vector, matrix, data frame, or even another list.

---

6. Other Important Data Structures

Matrix

A matrix organizes values into rows and columns:

```
x <- matrix(1:6, nrow = 2)

x
```

Useful functions:

```
dim(x)
nrow(x)
ncol(x)
rownames(x)
colnames(x)
is.matrix(x)
```

Data Frame

A data frame is one of the most important structures for data analysis.

```
students <- data.frame(
  name = c("John", "Sara", "David"),
  grade = c(90, 85, 95),
  passed = c(TRUE, TRUE, TRUE)
)
```

Each column can have a different data type.

Useful functions include:

```
dim(students)
nrow(students)
ncol(students)

names(students)
colnames(students)
rownames(students)

head(students)
tail(students)

str(students)
summary(students)

is.data.frame(students)
```

`View(students)` opens the data in an RStudio tab. That is often easier than printing a large table in the console.

A **tibble** is the tidyverse version of a data frame. Printing is cleaner, and character columns stay character instead of being turned into factors. `readr::read_csv()` returns a tibble.

```
library(tidyverse)

as_tibble(students)

tibble(
  name = c("John", "Sara", "David"),
  grade = c(90, 85, 95)
)
```

A tibble is still a data frame. `is.data.frame()` is `TRUE` for both.

Factor

A factor is commonly used to represent **categorical data**:

```
gender <- factor(c("Male", "Female", "Female", "Male"))
```

Useful functions:

```
levels(gender)
nlevels(gender)
table(gender)
is.factor(gender)
```

---

7. Comparisons and Logical Operators

A **logical** value is `TRUE` or `FALSE`. Comparison operators produce logicals:

```
13 > 4
# TRUE

13 < 4
# FALSE

4 >= 4
# TRUE

4 == 4
# TRUE

13 != 4
# TRUE
```

Use `==` to test equality. A single `=` is assignment or a named argument, not a comparison.

The operators are vectorized:

```
x <- c(1, 2, 3, 4)
y <- c(1, 4, 4, 4)

x == y
# TRUE FALSE FALSE TRUE

x < y
# FALSE TRUE TRUE FALSE
```

Combine conditions with `&` (and) and `|` (or):

```
TRUE & FALSE
# FALSE

TRUE | FALSE
# TRUE

(x < 3) & (y >= 4)
# FALSE TRUE FALSE FALSE

(x < 3) | (y >= 4)
# TRUE TRUE TRUE TRUE
```

`!` negates a logical. `xor(a, b)` is true when exactly one of `a` or `b` is true.

A comparison is the usual way to subset a vector:

```
x <- 1:5
x[x < 3]
# 1 2
```

`%%` is useful here. `n %% 3 == 0` is true when `n` is a multiple of 3.

---

8. Indexing

We often need part of an object, not the entire object.

The main indexing operator in R is `[ ]`.

Vectors

```
x <- c(10, 20, 30, 40)

x[1]
# 10

x[c(1, 4)]
# 10 40

x[2:3]
# 20 30
```

Positions in R start at 1, not 0.

Negative indices drop elements:

```
x[-1]
# 20 30 40
```

We can also index with a logical vector:

```
x[c(TRUE, FALSE, TRUE, FALSE)]
# 10 30
```

A common pattern is to keep the values that satisfy a condition:

```
x[x > 20]
# 30 40
```

If the elements have names, we can index by name:

```
scores <- c(midterm = 90, final = 85)

scores["final"]
# 85
```

Lists

For a list, `[` and `[[` do different things.

```
student <- list(
  name = "John",
  age = 25,
  grades = c(90, 85, 95)
)

student[1]
```

This returns a **list** of length 1.

```
student[[1]]
# "John"
```

This returns the element itself.

The `$` operator is a convenient way to extract a named element:

```
student$name
# "John"
```

A useful way to remember the difference is:

```
[   → a piece of the list, still a list
[[  → the element itself
$   → a named element
```

Matrices

A matrix is indexed by row and column:

```
x <- matrix(1:6, nrow = 2)

x[1, 2]
x[1, ]
x[, 2]
```

`x[1, ]` is the first row. `x[, 2]` is the second column.

Data Frames

A data frame can be indexed like a matrix and also like a list.

```
students$grade

students[1, ]

students[, "grade"]
```

`students$grade` extracts the `grade` column as a vector.

`students[1, ]` extracts the first row, still as a data frame.

An important detail:

```
students[, "grade"]
# a vector

students[, "grade", drop = FALSE]
# a data frame with one column
```

By default, extracting a single column drops the data-frame structure.

---

9. Functions

A **function** performs a particular task.

The general form is:

```
function_name(arguments)
```

For example:

```
sqrt(16)
# 4
```

Here:

- `sqrt` is the **function name**.
- `16` is an **argument**.
- `4` is the returned result.

Another example:

```
round(3.14159, digits = 2)
# 3.14
```

This function has multiple arguments.

You can read the documentation of a function using:

```
?round
```

or:

```
help(round)
```

You can also inspect the arguments:

```
args(round)
```

---

10. Positional and Named Arguments

Consider:

```
seq(from = 1, to = 10, by = 2)
```

The arguments have names:

```
from = 1
to   = 10
by   = 2
```

Because R can match arguments by their positions, we can also write:

```
seq(1, 10, 2)
```

Both produce:

```
# 1 3 5 7 9
```

Using argument names often makes code easier to understand.

A useful distinction is:

```
x <- 10                    # assignment: store a value

round(3.14159, digits = 2)   # = gives a name to an argument
```

`<-` creates an object. `=` inside a function call names an argument.

---

11. Default Arguments

Functions can provide **default values** for arguments.

For example:

```
round(3.14159)
# 3
```

We did not specify `digits`, so R uses its default.

Compare:

```
round(3.14159, digits = 2)
# 3.14
```

The help page of a function shows its available arguments and their defaults:

```
?round
```

---

12. The `seq()` Function

`seq()` is a useful example for understanding function arguments.

It creates sequences.

```
seq(from = 1, to = 10, by = 2)

# 1 3 5 7 9
```

Here:

```
from = 1    starting value
to   = 10   ending value
by   = 2    step size
```

We can also specify the desired number of elements:

```
seq(from = 1, to = 10, length.out = 4)

# 1 4 7 10
```

So there are different ways to tell `seq()` what sequence we want.

Some arguments in `seq()` have a default value of `NULL`. This brings us to several important special values in R.

---

13. Special Values: `NA`, `NaN`, `Inf`, and `NULL`

R has several special values that have different meanings:

Value	Meaning	Example
`NA`	Missing or unknown value	`c(10, NA, 20)`
`NaN`	Undefined numerical result ("Not a Number")	`0/0`
`Inf`	Infinity	`1/0`
`-Inf`	Negative infinity	`-1/0`
`NULL`	Absence of a value	`x <- NULL`

A simple way to remember them:

`NA` = I don't know the value.

`NaN` = The numerical calculation is undefined.

`Inf` = The numerical result is infinite.

`NULL` = There is no value at all.

---

14. Detecting Special Values: The `is.*()` Family

Consider:

```
x <- c(NA, NaN, Inf, 5)
```

Now apply several functions:

```
is.na(x)
# TRUE TRUE FALSE FALSE

is.nan(x)
# FALSE TRUE FALSE FALSE

is.infinite(x)
# FALSE FALSE TRUE FALSE

is.finite(x)
# FALSE FALSE FALSE TRUE
```

The results can be summarized as:

Value	`is.na()`	`is.nan()`	`is.infinite()`	`is.finite()`
`NA`	TRUE	FALSE	FALSE	FALSE
`NaN`	TRUE	TRUE	FALSE	FALSE
`Inf`	FALSE	FALSE	TRUE	FALSE
`5`	FALSE	FALSE	FALSE	TRUE

An important observation is:

```
is.na(NaN)
# TRUE

is.nan(NA)
# FALSE
```

Thus, `NaN` is also considered a missing value by `is.na()`.

---

15. Why Is `NULL` Different?

Consider:

```
x <- c(NA, NaN, Inf, NULL)

x
# NA NaN Inf
```

Although we supplied four things:

```
length(x)
# 3
```

Why?

Because `NULL` represents **the absence of a value**. It contributes no element to the vector.

Compare:

```
length(NA)
# 1

length(NaN)
# 1

length(Inf)
# 1

length(NULL)
# 0
```

Therefore:

`NA` is a missing value; `NULL` is no value.

We test for `NULL` using:

```
is.null(NULL)
# TRUE
```

---

16. Missing Values in Real Data

Suppose we have:

```
grades <- c(90, 85, NA, 95)
```

Then:

```
mean(grades)
# NA
```

Because one grade is unknown, R cannot calculate the ordinary mean.

Many R functions provide an `na.rm` argument:

```
mean(grades, na.rm = TRUE)
# 90
```

`na.rm` means:

```
NA remove
```

Other useful functions include:

```
is.na(grades)
```

to identify missing values,

```
anyNA(grades)
```

to determine whether there are any missing values,

and:

```
na.omit(grades)
```

to remove observations containing missing values.

For data frames, another important function is:

```
complete.cases()
```

For example:

```
students <- data.frame(
  grade = c(90, NA, 85),
  age = c(20, 21, NA)
)

complete.cases(students)
# TRUE FALSE FALSE
```

---

17. Useful `is.*()` Functions

R has a large family of functions beginning with `is.`. They ask questions about an object and usually return `TRUE` or `FALSE`.

Checking data types

```
is.numeric(x)
is.integer(x)
is.double(x)
is.character(x)
is.logical(x)
is.complex(x)
```

Checking data structures

```
is.vector(x)
is.list(x)
is.matrix(x)
is.array(x)
is.data.frame(x)
is.factor(x)
```

Checking special values

```
is.na(x)
is.nan(x)
is.finite(x)
is.infinite(x)
is.null(x)
```

These functions make the naming convention easy to remember:

```
is.numeric(x)
```

asks:

> "Is `x` numeric?"

and:

```
is.na(x)
```

asks:

> "Is `x` missing?"

---

18. Useful `as.*()` Functions

The `as.*()` family usually attempts to **convert** an object.

```
as.numeric(x)
as.integer(x)
as.double(x)

as.character(x)
as.logical(x)

as.vector(x)
as.list(x)
as.matrix(x)
as.data.frame(x)
as.factor(x)
```

This gives us a useful distinction:

```
is.*()  → asks what something is

as.*()  → tries to convert something
```

For example:

```
x <- "25"

is.numeric(x)
# FALSE

as.numeric(x)
# 25
```

---

19. A Useful Family of Functions to Remember

At this stage, students should recognize several families of functions.

Investigating an object

```
typeof()
class()
mode()
str()
length()
attributes()
names()
dim()
```

Asking what an object is

```
is.numeric()
is.integer()
is.character()
is.logical()

is.vector()
is.list()
is.matrix()
is.data.frame()
is.factor()

is.na()
is.nan()
is.infinite()
is.finite()
is.null()
```

Converting objects

```
as.numeric()
as.integer()
as.character()
as.logical()

as.vector()
as.list()
as.matrix()
as.data.frame()
as.factor()
```

Indexing

```
x[1]
x[1:3]
x[-1]
x[x > 20]
x["name"]

student[[1]]
student$name

m[1, 2]
df[1, ]
df$grade
```

Basic vector operations

```
c()
length()
seq()
rep()
sort()
rev()
unique()
duplicated()
```

Basic numerical summaries

```
sum()
mean()
median()
min()
max()
range()
var()
sd()
summary()
```

Missing-data functions

```
is.na()
anyNA()
na.omit()
complete.cases()
```

and the commonly encountered argument:

```
na.rm = TRUE
```

---

20. The Big Picture

The concepts introduced so far fit together:

```
VALUES
  │
  ├── Numeric
  ├── Character
  ├── Logical
  └── Other types
       │
       ▼
OBJECTS
       │
       ▼
DATA STRUCTURES
  │
  ├── Vector
  ├── List
  ├── Matrix
  ├── Data Frame
  └── Factor
       │
       ▼
INDEXING
  │
  ├── [    → select a part
  ├── [[   → extract an element
  └── $    → extract a named element
       │
       ▼
FUNCTIONS
  │
  ├── Function name
  ├── Arguments
  ├── Named/positional arguments
  └── Default arguments
       │
       ▼
SPECIAL VALUES
  │
  ├── NA    → missing/unknown value
  ├── NaN   → undefined numerical result
  ├── Inf   → infinity
  └── NULL  → absence of a value
```

Three function families are particularly useful to remember:

```
is.*()   → What is it?
as.*()   → Convert it
?        → How does it work?
```

For example:

```
is.numeric(x)
as.numeric(x)
?as.numeric
```

These ideas form the foundation for working with data in R.